Gaussian processes and the computation of Greeks

Bouke Hoekstra
Bruno Fernandez

June 2026

Contents

Abstract

This report explores the use of Gaussian processes (GPs) for pricing financial derivatives and computing their sensitivities, known as Greeks. We begin by introducing the foundations of Bayesian regression and how GPs extend this framework by placing a probability distribution directly over functions rather than parameters. We then discuss kernel selection, hyperparameter tuning, and scalable GP methods such as KISS-GP. In the experimental section, we compare six kernel choices on Black-Scholes call pricing and Greek computation, finding that Matérn kernels consistently outperform the standard RBF baseline. We further demonstrate how automatic differentiation via GPyTorch eliminates the need to derive kernel derivative formulas by hand. Finally, we compare KISS-GP against exact GP regression, showing that KISS-GP scales to datasets of 200,000 points where exact methods become infeasible.

1 Introduction

Options trading and risk management rely heavily on accurate computation of Greeks: the sensitivities of an option's price with respect to its underlying parameters. Greeks tell a trader how the value of their position changes as market conditions move, and are used to hedge against those movements. Errors in their computation translate directly into imperfect hedges and unintended risk exposures, making accuracy essential. In practice, these quantities are often computed either analytically, when a closed-form formula exists, or by finite differences on a pricing model. Both approaches have drawbacks: analytical formulas are model-specific and not always available, while finite differences can be slow and noisy. As financial models grow more complex and the demand for real-time risk management increases, there is a growing need for methods that are both fast and flexible.

Machine learning has emerged as a promising tool for addressing these challenges. Rather than deriving a new formula for every model, machine learning methods learn the structure of a pricing function directly from data, and can then be used to make fast predictions on new inputs. Among the many machine learning methods available, Gaussian processes stand out for a reason that is particularly valuable in finance: they do not just produce a single prediction, but also provide a measure of uncertainty around that prediction. In a field where the consequences of a wrong estimate can be significant, knowing how confident a model is in its output is often just as important as the output itself.

This report builds on and extends the GP examples presented in Dixon, Halperin, and Bilokon (2020) [2], which introduced the use of GPs for derivative pricing and Greek computation using a single RBF kernel. We extend this work in several directions: by comparing a broader set of kernel functions, by introducing automatic differentiation as an alternative to analytic kernel derivatives, and by benchmarking scalable GP methods that go well beyond the dataset sizes considered in the original examples.

The report is structured as follows. We begin in Chapter 2 by reviewing the foundations of regression, starting from classical linear regression and building up to Bayesian regression, which provides the probabilistic framework that GPs are built on. Chapter 3 introduces Gaussian processes formally, covering the role of the kernel function, the prediction equations, hyperparameter tuning, and scalable approximations such as KISS-GP. We also discuss several practical applications of GPs in finance. Chapter 4 then presents our experiments, where we apply GPs to the computation of Greeks under the Black-Scholes and Heston models, compare kernel choices, and benchmark the scalability of exact versus approximate GP training.

2 Regression

2.1 Classical Regression

Regression analysis is a statistical method used to describe and quantify relationships between variables. It is used to describe relationships between two variables or among several variables. For instance, rather than simply determining whether a patient has elevated blood pressure, one may wish to understand how factors such as age and weight influence the probability of developing high blood pressure. The variable being explained (blood pressure) is referred to as the dependent (or response) variable, while the variables used to explain it (age, weight) are known as independent (or predictor) variables. Regression analysis relies on a model that captures the relationship between the dependent and independent variables in a simplified mathematical form. In some cases, there may be justification for expecting a particular mathematical function to best represent such a relationship; in others, simplifying assumptions must be made (e.g., that blood pressure increases linearly with age).

Linear regression is used when the goal is to examine the linear relationship between a dependent variable Y (e.g., blood pressure) and one or more independent variables X (e.g., age, weight, sex). In this framework, the relationship is represented by a straight line according to the equation $Y = a + b \cdot X$. The parameter b, known as the regression coefficient, quantifies how much the independent variable X contributes to the dependent variable Y. First, the parameters a and b are estimated from observed values of X and Y using statistical methods, and the resulting regression line can then be used to predict the value of the dependent variable Y from that of the independent variable X.

In a fictitious study [10], data were collected from 135 participants: both women and men between the ages of 18 and 27, with heights ranging from 1.59 to 1.93 meters. The study investigated the relationship between height and weight, treating weight (in kilograms) as the dependent variable to be predicted from height (in centimeters) as the independent variable. From this data, the following regression equation was derived: Y= –133.18 + 1.16 × X, where X is height in centimeters and Y is weight in kilograms. The regression coefficient of 1.16 indicates that, according to this model, each additional centimeter of height corresponds to an increase in body weight of 1.16 kg.

This was an example of a univariable linear regression model, which models the linear relationship between the dependent variable Y and a single independent variable X. However, in many cases the dependent variable Y depends on several factors. In this case, we can use a multivariable regression model to study the effect of multiple variables on the dependent variable. A multivariable linear regression model looks like: $Y = a + b_1 \cdot X_1 + \cdots + b_n \cdot X_n$

2.2 Bayesian Regression

Classical regression seeks a single fixed set of parameters: weights, that best fit the observed data. Bayesian regression takes a fundamentally different stance: rather than treating the model parameters as unknown constants to be estimated, it treats them as **random variables** provided with a probability distribution. This distribution reflects both what you knew before collecting data and what the data tells you.

The typical Bayesian workflow consists of three main steps: capturing available knowledge about a given parameter in a statistical model via the prior distribution, which is typically determined before data collection; determining the likelihood function using the information about the parameters available in the observed data; and combining both the prior distribution and the likelihood function using Bayes' theorem to obtain the posterior distribution. [11]

To make this concrete, consider the height and weight example from the previous section. In classical regression, we arrived at a single fixed estimate: each additional centimeter of height corresponds to exactly 1.16 kg of additional weight. In the Bayesian framework, rather than committing to this single value, we place a prior distribution over the slope and update it using the observed data from the 135 participants. The result is a posterior distribution over the slope. We might conclude that the slope is most likely around 1.16, but that there is a 95% probability it lies within some interval around that value. This uncertainty also carries through to predictions: rather than predicting a single weight for a new person given their height, the Bayesian model produces a full predictive distribution. Crucially, this uncertainty grows for heights that were rare or absent in the original dataset.

2.2.1 Prior Distribution

The starting point is a **prior distribution** $p(\theta)$, which captures beliefs about plausible parameter values before any data is seen. Prior distributions play a defining role in Bayesian statistics. Priors can come in many different distributional forms, such as a normal, uniform or Poisson distribution, among others. The process by which a suitable prior distribution is constructed is called *Prior Elicitation*. This can be done for example based on previous data, or by asking experts to provide estimates in this regard.

The individual parameters that control the amount of uncertainty in the priors are called *hyperparameters*. Take a normal prior as an example. This distribution is defined by a mean and a variance that are the hyperparameters for the normal prior, and we can write this distribution as $\mathcal{N}(\mu_0, \sigma_0^2)$, where μ_0 represents the mean and σ_0^2 represents the variance. A larger variance represents a greater amount of uncertainty surrounding the mean, and vice versa.

Priors can have different levels of informativeness; the information reflected in a prior

distribution can be anywhere on a spectrum from complete uncertainty to relative certainty. An informative prior is one that reflects a high degree of certainty about the model parameters being estimated. For example, an informative normal prior would be expected to have a very small variance. A diffuse (non-informative) prior reflects a great deal of uncertainty about the model parameter. This prior form represents a relatively flat density and does not include specific knowledge of the parameter . A researcher may want to use a diffuse prior when there is a complete lack of certainty surrounding the parameter. In this case, the data will largely determine the posterior.

Overall, there is no right or wrong prior setting. Many times, diffuse priors can produce results that are aligned with the likelihood, whereas other times inaccurate or biased results can be obtained with relatively flat priors. Likewise, an informative prior that does not overlap well with the likelihood can shift the posterior away from the likelihood, indicating that inferences will be aligned more with the prior than the likelihood. Regardless of the informativeness of the prior, it is always important to conduct a prior sensitivity analysis to fully understand the influence that the prior settings have on posterior estimates.

The subjectivity of priors is highlighted by critics as a potential drawback of Bayesian methods. However, priors are typically defined through previous beliefs, information or knowledge, therefore it is not completely subjective in our eyes.

2.2.2 Likelihood Function

The second component of Bayesian regression is the **likelihood function** $p(\mathbf{y} \mid \mathbf{X}, \theta)$, which captures what the observed data tell us about the parameters.The likelihood function answers the question: "Given a specific set of model parameters, what is the provabability of observing our actual data?" The likelihood function quantifies the strength of support the observed data lends to possible values for the unknown parameters θ.

A key distinction between Bayesian and classical regression lies in how the likelihood is used. In classical regression, the unknown parameters are considered to be fixed constants, and the likelihood is the conditional probability of the data given those fixed parameters. In Bayesian regression, however, the unknown parameters are treated as random variables. The observed data are treated as fixed, while the parameter values are varied. Therefore, the likelihood function summarises three elements: a statistical model that stochastically generates the data, a range of possible values for θ, and the observed data $\mathbf{y}$.

In some cases, specifying a likelihood function can be very straightforward. Consider the following simple example: suppose we are given a coin and want to know the probability of obtaining heads, denoted θ. We toss the coin n times and count the number of heads, where y is the total number of heads observed. Assuming the probability of heads remains constant across all flips, the probability of the observed number of heads is given

by the binomial distribution:

$$p(y \mid \theta) = \binom{n}{y} \theta^y (1-\theta)^{n-y}, \quad 0 \leq \theta \leq 1. \tag{1}$$

When y is kept fixed and θ is varied, $p(y \mid \theta)$ becomes a continuous function of θ, known as the **binomial likelihood function**. For example, if the coin is flipped 10 times and 4 heads are observed, the likelihood function becomes:

$$p(y \mid \theta) = \binom{10}{4} \theta^4 (1-\theta)^6, \quad 0 \leq \theta \leq 1. \tag{2}$$

This example illustrates how, even in a simple setting, the likelihood function cleanly expresses how probable the observed data are for each possible value of the unknown parameter θ.

2.2.3 Posterior Distribution

The **posterior distribution** $p(\theta \mid \mathbf{y})$ is the central output of Bayesian regression. It represents one's updated knowledge about the parameters after combining the prior distribution with the evidence from the observed data. In other words, it answers the question: *"Given what we believed before about the parameters, and given what the data shows, what is the probabilty of the parameters?"*.

The posterior distribution is obtained through **Bayes' theorem**, which provides the mathematical rule for updating beliefs in light of new data. For a dataset $\mathbf{y}$ and model parameters θ, Bayes' theorem is written as:

$$p(\theta \mid \mathbf{y}) = \frac{p(\mathbf{y} \mid \theta)\, p(\theta)}{p(\mathbf{y})}, \tag{3}$$

which is often simplified to the proportional form:

$$p(\theta \mid \mathbf{y}) \propto p(\mathbf{y} \mid \theta)\, p(\theta). \tag{4}$$

Here, $p(\theta \mid \mathbf{y})$ is the posterior distribution, $p(\mathbf{y} \mid \theta)$ is the likelihood function, $p(\theta)$ is the prior distribution, and $p(\mathbf{y})$ is a normalising constant that ensures the posterior integrates to one. Using this we see that the posterior distribution is proportional to the likelihood function multiplied by the prior distribution.

The posterior distribution reflects a balance between prior knowledge and the evidence in the data. When the prior is diffuse, the posterior is largely driven by the likelihood, so by the data. When the prior is strongly informative, it can pull the posterior away from what the data alone would suggest. This balance is illustrated in Figure 1, which shows how different prior settings lead to different posterior distributions, when the likelihood

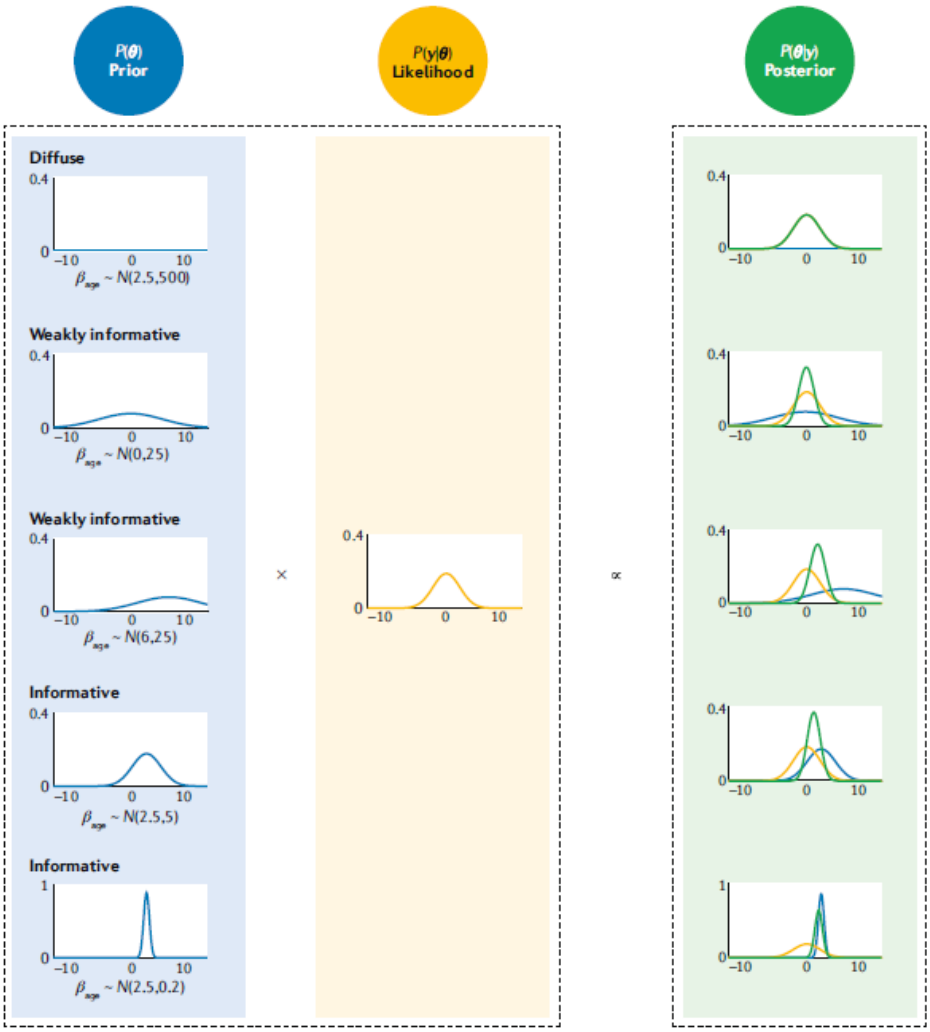

Fig. 2 | **Illustration of the key components of Bayes' theorem.** The prior distribution (blue) and the likelihood function (yellow) are combined in Bayes' theorem to obtain the posterior distribution (green) in our calculation of PhD delays. Five example priors are provided: one diffuse, two weakly informative with different means but the same variance and two informative with the same mean but different variances. The likelihood remains constant as it is determined by the observed data. The posterior distribution is a compromise between the prior and the likelihood. In this example, the posterior distribution is most strongly affected by the type of prior: diffuse, weakly informative or informative. β_{age}, linear effect of age (years); θ, unknown parameter; $P(.)$, probability distribution; y, data.

Figure 1: Example posterior distribution. Source: [10]

remains fixed.

Rather than producing a single point estimate for each parameter, as classical regression does, Bayesian regression produces a full probability distribution over the parameters. This posterior distribution is often summarised using a point estimate such as the posterior mean or median, together with a **credible interval**. A 95% credible interval means there is a 95% probability that the true parameter value lies within that interval.

Direct analytical computation of the posterior distribution is usually not possible, because the mathematical expression describing it is typically both complex and high-dimensional. The denominator $p(\mathbf{y})$ in Bayes' theorem is in general not available in

closed form, as it requires integrating over all possible parameter values:

$$p(\mathbf{y}) = \int p(\mathbf{y} \mid \theta)\, p(\theta)\, d\theta. \tag{5}$$

This integral is often analytically intractable. As a result, computational methods are often used to approximate the posterior distribution. The most widely used approach is **Markov chain Monte Carlo (MCMC)**, which works by repeatedly sampling plausible parameter values and gradually building up an empirical approximation of the full posterior distribution.

Although Bayesian regression produces a full posterior distribution rather than a single estimate, it is sometimes useful to summarise the posterior with a single point. The **Maximum A Posteriori (MAP) estimate** is the value of θ that maximises the posterior distribution, so it is the most probable parameter value given both the data and the prior. For a Gaussian posterior, the MAP estimate coincides with the posterior mean.

Once the posterior distribution over the parameters has been obtained, Bayesian regression can be used to make predictions for new, previously unseen data points. This is one of the key strengths of the Bayesian approach: predictions do not come as a single number, but as a full **predictive distribution**, which naturally captures how uncertain the model is about its prediction.

Concretely, to predict the output f_* at a new input x_*, Bayesian regression averages the model's prediction over all possible parameter values, weighted by how probable each parameter value is under the posterior. This process of integrating out the parameters means the prediction is no longer dependent on any single set of weights, but reflects the full uncertainty in the model. The resulting predictive distribution is Gaussian:

$$f_* \mid x_*, \mathcal{D} \sim \mathcal{N}(\mu_*, \Sigma_*), \tag{6}$$

where $\mathcal{D}$ denotes the observed training data, μ_* is the predicted mean, and Σ_* is the predicted variance. The mean μ_* gives the most likely prediction, while the variance Σ_* quantifies the uncertainty around that prediction.

An important property of Bayesian prediction is that the predictive uncertainty naturally increases the further one predicts from the observed training data. Where data is dense, the model is confident; where data is sparse, the uncertainty grows. This is a significant advantage over classical regression, which produces the same type of point prediction regardless of how much data supports it.

3 Gaussian Processes

Gaussian Processes (GP's) are an extension of the Bayesian regression framework introduced in the previous chapter. While Bayesian regression places a probability distribution over a fixed set of parameters, a Gaussian Process takes this idea one step further: instead of placing a distribution over parameters, it places a distribution directly over functions.. Rather than specifying a parametric form $f_\theta(x)$ and placing a distribution on θ, a GP places a probability distribution over the space of all possible functions from inputs to outputs. In other words, rather than asking "what are the most likely parameter values?", a GP asks "what is the most likely function that explains the data?" Formally, a random function $f : \mathbb{R}^p \to \mathbb{R}$ is said to be drawn from a GP,

$$f \sim \mathcal{GP}(\mu, k) \tag{7}$$

if, for any finite collection of input points $x_1, \ldots, x_n$, the corresponding function values are jointly Gaussian [2]:

$$\big[f(x_1), \ldots, f(x_n)\big] \sim \mathcal{N}(\mu_X,\, K_{X,X}) \tag{8}$$

A GP defines a probability distibution over all such possible functions f. This makes GPs a non-parametric method, meaning the model does not assume a fixed functional form. The model complexity can grow with the data, making it far more flexible than standard linear regression.

3.1 Mean and Kernel function

The GP is fully specified by two ingredients:

1. **mean function** $\mu(x)$: The mean function represents the expected value at any input point x. Often in practical settings, the mean function is simply set to zero before seeing any data.

2. **kernel function** $k(x, x')$: The kernel (covariance) function controls how similar we expect the function values to be at two different input points x and x'. The covariance function assigns a high (correlation) between two input points, if their function values are expected to behave similary at nearby points. If nearby input points behave very differently in the function, the correlation is low. The kernel function encodes important properties about the structure of f such as for example its smoothness and how quickly correlations decay with distance in input space. The kernel function defines the covariance matrix $K_{X,X}$ via $(K_{X,X})_{ij} = k(x_i, x_j)$.

A function drawn from a gaussion process is then written as:

$$f \sim \mathcal{GP}(\mu(x), k(x, x')) \tag{9}$$

A commonly used kernel is the **Squared Exponential (SE)** kernel, also known as the RBF kernel,

$$k(x, x') = \exp\left\{-\frac{\|x - x'\|^2}{2\ell^2}\right\} \tag{10}$$

where the length-scale ℓ controls how quickly function values become uncorrelated with distance. A large length-scale produces smooth, slowly varying functions, whereas a small length scale allows the function to vary rapidly.

The SE kernel is infinitely smooth: functions drawn from a GP with an SE kernel are differentiable infinitely many times. This makes it a natural default choice when the underlying function is expected to vary smoothly and gradually. Furthermore, we see that the SE kernel is stationary, meaning the covariance between two points depends only on the distance between them, not on where they are located in input space. Two points that are close together will always have a high covariance.

Another commonly used kernel is the **Matérn kernel** [9], introduced by Rasmussen and Williams (2006) as a more flexible alternative to the SE kernel. The key limitation of the SE kernel is that it assumes the underlying function is infinitely smooth, which is often unrealistic in practice. The Matérn class addresses this by introducing an additional parameter $\nu > 0$ that directly controls the smoothness of the function: a GP with a Matérn kernel is exactly k-times mean-square differentiable if and only if $\nu > k$. The general form of the Matérn kernel is:

$$k_{\text{Matérn}}(x, x') = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu}\,\|x - x'\|}{\ell}\right)^{\nu} K_\nu \left(\frac{\sqrt{2\nu}\,\|x - x'\|}{\ell}\right), \tag{11}$$

where ℓ is the length-scale, Γ is the gamma function, and K_ν is a modified Bessel function. The two most practically useful special cases are $\nu = 3/2$ and $\nu = 5/2$, which simplify to:

$$k_{\nu=3/2}(r) = \left(1 + \frac{\sqrt{3}\,r}{\ell}\right) \exp\left(-\frac{\sqrt{3}\,r}{\ell}\right), \tag{12}$$

$$k_{\nu=5/2}(r) = \left(1 + \frac{\sqrt{5}\,r}{\ell} + \frac{5r^2}{3\ell^2}\right) \exp\left(-\frac{\sqrt{5}\,r}{\ell}\right). \tag{13}$$

where $\mathrm{r} = \|x - x'\|$. These produce functions that are once and twice differentiable respectively, making them more suitable than the SE kernel when the true underlying function is expected to be moderately smooth. Note that as $\nu \to \infty$, the Matérn kernel

converges to the SE kernel, so the SE kernel can be seen as the limiting case of infinite smoothness.

Beyond choosing a single kernel, it is also possible to construct new, more expressive kernels by combining simpler ones. Two operations are particularly useful: the **sum** and the **product** of two kernels. If k_1 and k_2 are both valid kernel functions, then both $k_1(x, x') + k_2(x, x')$ and $k_1(x, x') \cdot k_2(x, x')$ are also valid kernels. This means that complex behaviour can be modelled by combining simple building blocks. For example, adding a periodic kernel to a smooth kernel produces a function that is both smooth and periodic, which could be useful for modelling seasonal patterns in finance for example.. As another example, the **Rational Quadratic (RQ)** kernel,

$$k_{\text{RQ}}(r) = \left(1 + \frac{r^2}{2\alpha\ell^2}\right)^{-\alpha}, \tag{14}$$

can be interpreted as an mixture of SE kernels with different length-scales.

The SE kernel commits to a single length-scale, meaning it assumes the function varies at roughly one particular speed. The RQ kernel extends this by effectively averaging over many SE kernels with different length-scales simultaneously. Some of those SE kernels have short length-scales (capturing rapid local variation) and some have long length-scales (capturing slow global trends), all at once. This makes the RQ kernel more flexible when the function behaves differently at different scales. In the limit $\alpha \to \infty$, the RQ kernel converges to the SE kernel.

Figure 2 illustrates sample paths of one-dimensional GP's under four different kernel functions: SE, RQ, Matern 3/2 and Matern 5/2. As you can see the SE kernel produces the smoothest sample paths, while the Matérn kernels generate progressively rougher functions as the parameter ν decreases. Furthermore the RQ kernel produces a smooth function where the smoothness varies throughout the function.

The choice of kernel ultimately depends on the prior beliefs about the structure of the function being modelled, and different kernels can always be combined to build a richer model when a single kernel is not sufficient.

3.2 GP prediction

Given training data, GP regression works by conditioning the prior distribution over functions on the observed data points. You start with a wide range of possible functions consistent with the prior, and then progressively rule out those that disagree with the observations.

Given training inputs $x \in X$ and training targets $y \in Y$ with additive Gaussian noise $y \mid x \sim \mathcal{N}(f(x), \sigma_n^2)$, i.e $y_i = f(x_i) + \epsilon_i$ where $\epsilon_i \sim \mathcal{N}(0, \sigma_n^2)$. The posterior predictive distribution at test inputs $x_* \in X_*$ is:

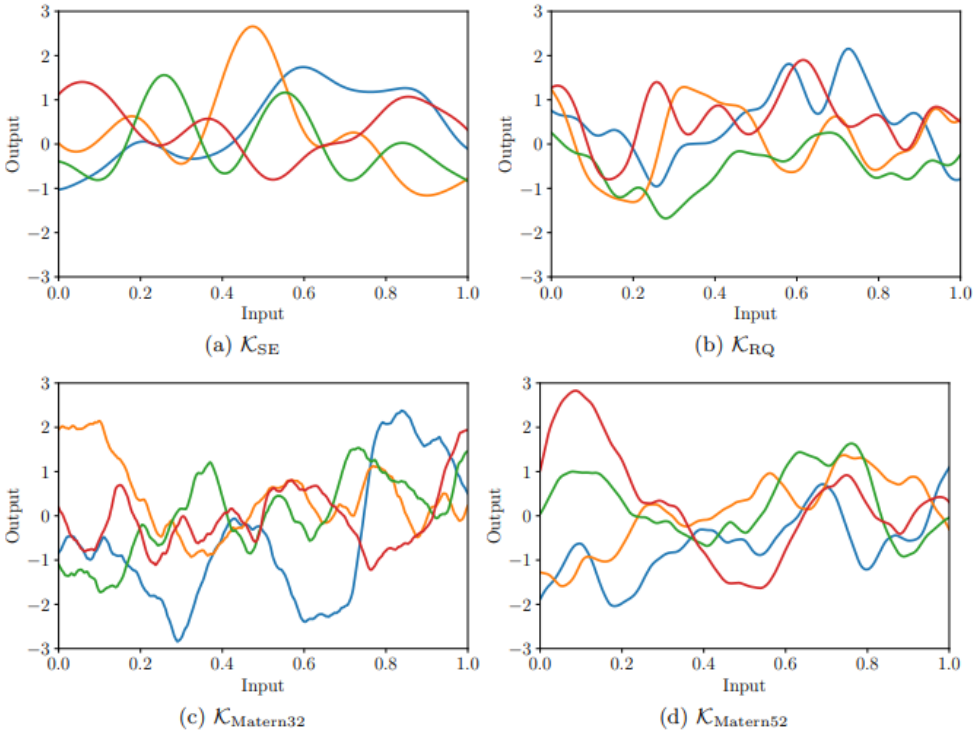

Figure 2: Sample paths of GP under different kernels

$$f_* \mid X, Y, x_* \sim \mathcal{N}(\mathbb{E}[f_* \mid X, Y, x_i],\ \mathrm{Var}[f_* \mid X, Y, x_i]) \tag{15}$$

with

$$\mathbb{E}[f_* \mid X, Y, X_*] = K_{X_*,X}\left[K_{X,X} + \sigma_n^2 I\right]^{-1} Y \tag{16}$$

$$\mathrm{Var}[f_* \mid X, Y, X_*] = K_{X_*,X_*} - K_{X_*,X}\left[K_{X,X} + \sigma_n^2 I\right]^{-1} K_{X,X_*} \tag{17}$$

The mean gives the most likely prediction and the variance naturally quantifies the uncertainty around that prediction. Of course, this uncertainty is larger in regions where little training data is available, and smaller near observed data points. This uncertainty quantification is one of the most important practical advantages of GP regression over classical regression.

To make the GP framework concrete, consider a simple one-dimensional regression problem. We use a zero mean function and the squared exponential kernel introduced above, with length-scale $\ell = 1$:

$$k(x_p, x_q) = \exp\left(-\tfrac{1}{2}|x_p - x_q|^2\right). \tag{18}$$

Together, these two ingredients fully specify a GP prior over functions. Figure 3(a) shows

three functions sampled from this prior. The shaded region shows the 95% pointwise confidence band; because no data has been seen yet, this band is the same width everywhere, and is very uncertain. Figure 3(b) shows the result after conditioning on five observations. Following the steps described in the previous section, we update the prior using the observed data to obtain a posterior distribution over functions. The solid line is the posterior mean and the shaded region is again the 95% confidence band. The posterior mean passes through every observation, and the confidence band collapses near the data points. Away from the data, where no observations have been made, the uncertainty grows back towards the prior level

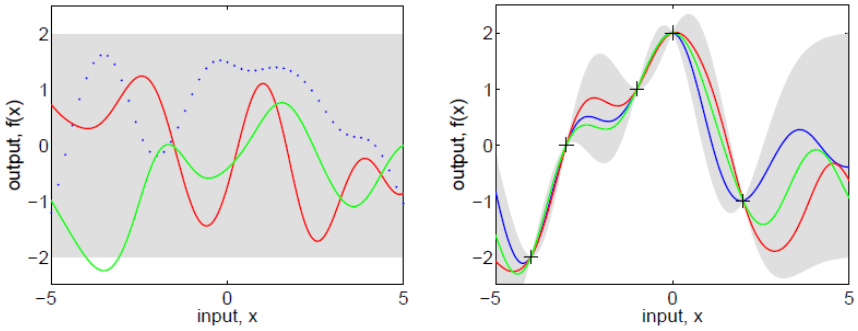

Figure 3: GP regression with a squared exponential kernel

3.3 Choosing Hyperparameters

The kernel functions typically contain adjustable parameters: the hyperparameters. The kernel hyperparameters (e.g., ℓ and σ_n) are usually not set manually but are learned from the data. This is done by maximising the **marginal likelihood** (evidence), which is the probability of the observed data given the model.:

$$\log p(Y \mid X, \lambda) = -\left[Y^\top (K_{X,X} + \sigma_n^2 I)^{-1} Y + \log\det(K_{X,X} + \sigma_n^2 I)\right] - \frac{n}{2}\log 2\pi \qquad (19)$$

Here, the first term measures how well the model fits the data, the second term penalises model complexity, and the third term is just a normalisation constant. Therefore, maximising the evidence, i.e. computing $\lambda_* = argmax_\lambda \ log\ p(y|x,\lambda)$, automatically balances model fit against model complexity: a simpler model is preferred unless the data strongly supports a more complex one. This is sometimes referred to as "Occam's razor.". In practice, the negative evidence is minimised using stochastic gradient descent using the analytically available gradient.

Figure 4 illustrates the effect of choosing different hyperparameter values on the GP predictions. The data points, indicated by + symbols, were generated from a GP using

the squared exponential kernel in one dimension: with varying hyperparameters, and the shaded region represents the 95% confidence interval for the underlying function f.

Figure 4(a) shows the predictions using the correct hyperparameters. Notice how the uncertainty grows naturally in regions far from any training points, and how the model provides a smooth and sensible fit to the data. Figure 4(b) shows the result of using a length-scale that is too short ($\ell = 0.3$). Because the length-scale is small, the model assumes that function values become uncorrelated very quickly as inputs move apart. As a result, the model explains the data through sharp, rapid variations, and we see that the uncertainty grows very rapidly away from each training point, since nearby points provide almost no information about the function value further away. Figure 4(c) shows the opposite situation, where the length-scale is too long ($\ell = 3$). Now the model assumes function values remain highly correlated even for inputs that are far apart, forcing the underlying function to vary very slowly. To reconcile this with the spread in the observed data, the noise level is increased substantially meaning the model attributes most of the variation in the data to noise rather than to the function itself. This illustrates why maximising the marginal likelihood is an effective strategy for hyperparameter tuning: it automatically identifies the hyperparameter values that best balance model fit and model complexity, without requiring any manual tuning.

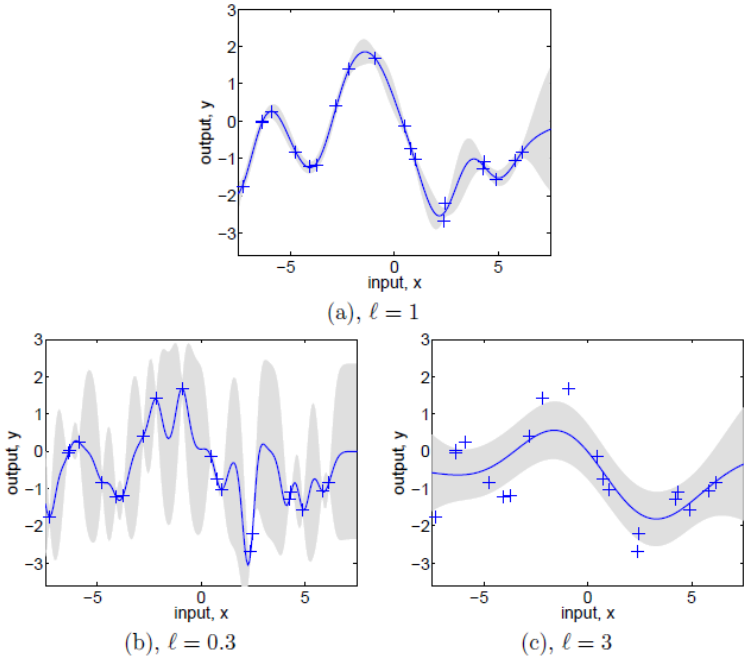

Figure 4: Data generated from a GP with different hyperparameters

The principal computational cost of GP regression is the training step required for maximizing the marginal likelihood with respect to the kernel hyperparameter. Specifically, to do this we need the inversion of the $n \times n$ covariance matrix $K_{X,X}$, which requires $\mathcal{O}(n^3)$ time via Cholesky decomposition (where n is the number of training data points).

This makes standard GP's impractical for large datasets. Once this is computed (once training is complete), predictions themselves reduce to $\mathcal{O}(n^2)$ matrix vector multiplications.

3.4 Massively scalable Gaussian processes

As discussed in the previous section, the primary computational bottleneck of standard GP regression is the $\mathcal{O}(n^3)$ cost of the Cholesky decomposition of the $n \times n$ covariance matrix $K_{X,X}$, which must be performed during training. For large datasets this quickly becomes impractical. Two complementary strategies have been developed to break this barrier: structured kernel approximations that exploit algebraic regularities in $K_{X,X}$, and kernel approximations that replace the exact kernel by a cheaper surrogate. We discuss both in turn.

3.4.1 KISS-GP

Wilson and Nickisch (2015) [12] introduced a method called **Kernel Interpolation for Scalable Structured Gaussian Processes (KISS-GP)** to address this bottleneck. The core idea is that rather than working with the full $n \times n$ covariance matrix $K_{X,X}$ directly, KISS-GP replaces it with a structured approximation that is much cheaper to work with. This is done by introducing $m \ll n$ **inducing points** placed on a regular Cartesian grid U, and approximating the full covariance matrix as:

$$K_{X,X} \approx W_X \, K_{U,U} \, W_X^\top, \tag{20}$$

where $K_{U,U}$ is the $m \times m$ covariance matrix between the inducing points, and $W_X \in \mathbb{R}^{n \times m}$ is a sparse matrix of interpolation weights that maps each training point to its nearest inducing point neighbours. Since $m \ll n$, working with $K_{U,U}$ is already much cheaper than working with the full $K_{X,X}$. KISS-GP goes one step further by exploiting the algebraic structure that arises when the inducing points are placed on a regular grid.

Specifically, for a stationary kernel such as the RBF kernel, $K_{U,U}$ decomposes as a **Kronecker product** of **Toeplitz matrices**:

$$K_{U,U} = T_1 \otimes T_2 \otimes \cdots \otimes T_P, \tag{21}$$

where P is the number of input dimensions and each T_d is a Toeplitz matrix along the d-th axis. A Toeplitz matrix has the special property that each row is a shifted version of the previous row, which means it can be diagonalised using the **Fast Fourier Transform (FFT)** in only $\mathcal{O}(m \log m)$ time rather than $\mathcal{O}(m^3)$. The Kronecker structure further allows the full matrix to be decomposed dimension by dimension, each of which is much smaller than $K_{U,U}$ itself, bringing the matrix-vector product cost down to $\mathcal{O}(Pm^{1+1/P})$.

Rather than inverting $K_{U,U}$ explicitly, KISS-GP replaces the Cholesky decomposition with a **conjugate gradient solver**, which solves the required linear system iteratively. Crucially, each iteration of the conjugate gradient solver only requires a matrix-vector product and never needs to form or invert the full matrix explicitly. Because W_X is sparse and $K_{U,U}$ has the Kronecker and Toeplitz structure described above, each such matrix-vector product costs only $\mathcal{O}(n + m \log m)$. Combined with the conjugate gradient solver, this brings the overall training complexity down to $\mathcal{O}(n)$, near-linear in the number of training points, compared to the $\mathcal{O}(n^3)$ cost of standard GP training.

One practical limitation of KISS-GP is that the grid size grows exponentially in the number of input dimensions, $m = k^p$, so KISS-GP works best for low-dimensional problems with $p \lesssim 4$. This situation matches many financial applications naturally, where the relevant inputs are (S, K, T, r, σ).

3.4.2 Other Kernel Approximations

Beyond KISS-GP/SKI-GP, several other approaches exist for scaling GP regression to large datasets. The **Nyström approximation** selects m landmark points and approximates the full kernel matrix as $K \approx K_{nm} K_{mm}^{-1} K_{mn}$, reducing the computational cost to $\mathcal{O}(m^2 n)$. The quality of this approximation depends heavily on how the landmark points are chosen.

Random Fourier Features [8] take a different approach, exploiting Bochner's theorem [6] which states that any stationary kernel can be written as the Fourier transform of a probability density $p(\boldsymbol{\omega})$. By sampling M random frequencies $\boldsymbol{\omega}_1, \ldots, \boldsymbol{\omega}_M \sim p$, one can construct explicit feature vectors

$$\mathbf{z}(\mathbf{x}) = \sqrt{\frac{2}{M}} \left[\cos\left(\boldsymbol{\omega}_j^\top \mathbf{x} + b_j\right)\right]_{j=1}^{M}, \tag{22}$$

such that $\mathbf{z}(\mathbf{x})^\top \mathbf{z}(\mathbf{x}') \approx k(\mathbf{x}, \mathbf{x}')$. This reduces GP regression to a Bayesian linear regression in M features at a cost of $\mathcal{O}(nM^2 + M^3)$, which is tractable when $M \ll n$.

3.5 Advantages of Gaussian Processes

Gaussian processes are widely used, because of a few important characteristics. They inherit all the advantages of the Bayesian regression framework introduced in the previous chapter, and extend them in several ways.

To begin with, just like with Bayesian Regression, a GP does not produce a single predicted value for new data points, but instead produces a full predictive distribution over function values. This distribution comes with a natural uncertainty estimate which quantifies how confident the model is in its prediction at any given input point. This is particularly valuable in high-stakes applications such as finance.

Furthermore, GPs allow prior knowledge about the structure of the underlying function to be formally incorporated into the model through the choice of kernel function and mean function. The kernel encodes beliefs about properties such as the smoothness of the function and how quickly correlations decay with distance in input space. For example, if one believes the option price varies smoothly, this can be encoded directly through the choice of kernel before seeing any data. This is a more flexible form of prior knowledge than what is available in standard Bayesian regression, where prior knowledge is limited to beliefs about a fixed set of parameters.

Moreover, because the kernel already encodes structural information about the function, a GP can produce sensible and smooth predictions even when only a small number of training points are available. The prior contributes information alongside the data, meaning the model does not need to rely on the data alone to produce reliable estimates. This is clearly illustrated in our experiments, where a GP trained on as few as 225 price observations is able to recover the correct Greeks with high accuracy.

Lastly, unlike standard Bayesian regression which assumes a fixed parametric form for the function being modelled, a GP places a distribution directly over the space of all possible functions. This makes GPs a non-parametric method: the model does not commit to any particular functional form in advance, and its complexity grows naturally with the data.

3.6 Gaussian Processes in Finance

There are many applications for Gaussian Processes in Finance, where the ability to produce uncertainty estimates alongside predictions are very valuable. GP's have been applied across a wide range of financial problems, including risk management, asset management, derivative pricing and market making. Two concrete examples that illustrate this potential are: yield curve modeling and the calibration of trend-following strategies.

One application is fitting and forecasting the yield curve. The yield curve is a graph that shows the relationship between interest rates and the time to maturity of bonds. It shows how much return an investor gets for lending money over different time periods, for example lending 1 month vs 1 year vs 10 years. Under normal circumstances the yield curve slopes upward, meaning longer-term bonds offer higher interest rates than short-term ones. This makes intuitive sense: if you lend money for a longer period, you take on more risk and uncertainty, so you expect a higher return in compensation. Gonzalvez et al. [5] use a GP trained on daily US yield data across nine maturities to fit the yield curve and forecast interest rates. On each trading day, the model used all data observed to that point to produce a one-day ahead prediction of the interest rate. These predictions are then compared against the rates that were actually observed. Figure 5 shows this rolling prediction for the interest rate with 2-year maturity throughout 2016, where the solid line

is the GP prediction, the dashed line is the actual observed rate, and the shaded region is the 95% confidence interval. The key advantage over traditional approaches is that the GP does not just produce a single prediction, but also provides a confidence interval around it.

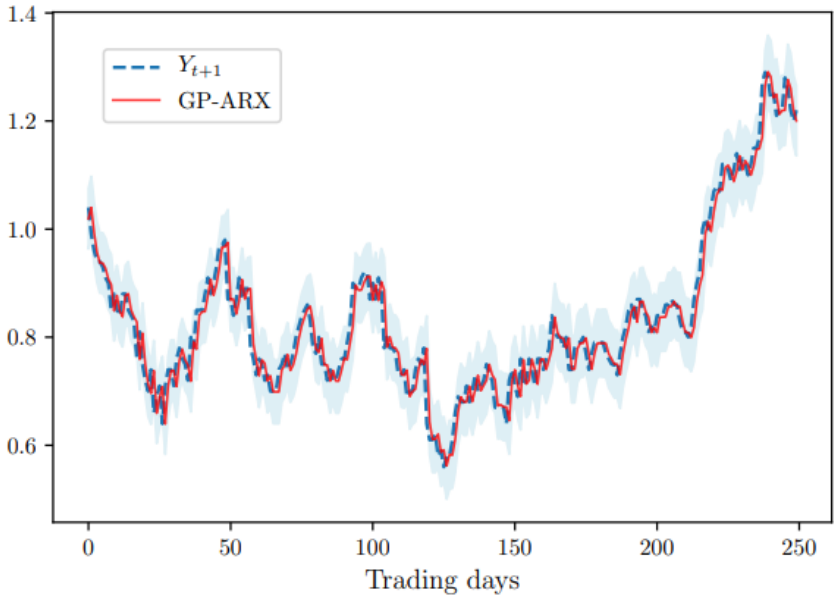

Figure 5: One-day-ahead GP prediction of the 2-year U.S. spot rate during 2016, [5]

A second application concerns trend-following strategies. The idea behind a trend-following strategy is simple: bet on assets that have been going up and against assets that have been going down. In practice, this means constructing a portfolio of assets where the expected return of each asset is estimated by looking at how its price has moved over some recent period. The strategy depends on three key parameters:

1. The trend window : the window length used to estimate the trend (i.e. how far back to look when estimating expected returns)

2. The risk window: this controls how far back you look when estimating how risky each asset is, i.e how much its price tends to fluctuate

3. The turnover penalty: this controls how much the portfolio is allowed to change between two rebalancing dates. A high penalty means the portfolio changes very little from one period to the next, while a low penalty means the portfolio can change a lot, reacting more aggressively to new information.

In a classical approach, these parameters are fixed in advance and kept constant throughout time. The optimal portfolio is then found given these fixed parameters. Gonzalvez et al. [5] take a fundamentally different approach. Rather than fixing the parameters in advance, they solve both problems simultaneously: at each rebalancing date, they search for

the parameter values and the portfolio weights that together give the best result. This is done using Bayesian optimization, which uses a GP internally to efficiently search through the space of possible parameter combinations. The reason a GP is needed for this is that evaluating the quality of a given set of parameters is expensive, because the gradient of the objective function with respect to the parameters is not available. The GP acts as a cheap surrogate for this expensive evaluation, allowing the optimizer to intelligently decide which parameter combinations are worth trying next. The dataset consists of daily prices of 13 futures contracts on worldwide equity indices and 10-year sovereign bonds from 2006 to 2017. Figure 6 shows that the Bayesian optimized strategy outperforms the fixed-parameter strategy over this period, improving the annualized Sharpe ratio from 1.56 to 1.71 and reducing the maximum drawdown from 5.7% to 4.7%.

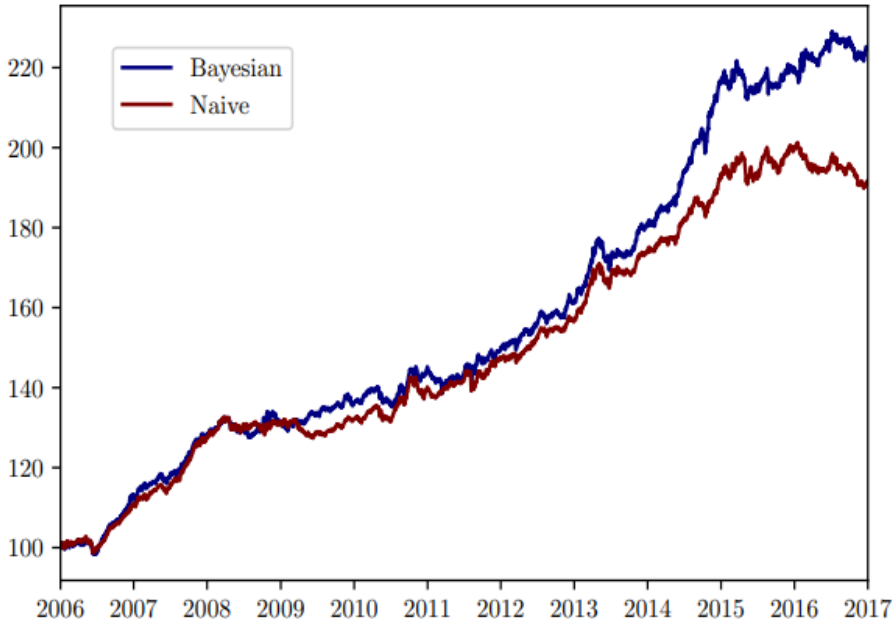

Figure 6: Cumulative performance of the Bayesian optimized trend-following strategy versus the fixed-parameter strategy. Source: [5]

Here the Sharpe ratio is a measure of how good a strategy is, taking into account both the return it generates and the risk it takes to generate that return. A higher Sharpe ratio means you are getting more return per unit of risk, and the Maximum Drawdown measures the worst loss you would have suffered if you had invested in the worst possible time. So when the Bayesian strategy improved the Sharpe ratio from 1.56 to 1.71 and reduced the maximum drawdown from 5.7% to 4.7%, this means that the dynamic parameter approach both generated better risk-adjusted returns and was less prone to large losses than the fixed-parameter approach.

These two examples illustrate how Gaussian processes can be applied to real financial

problems. In the remainder of this report, we shift focus to a third application: the use of Gaussian processes for the computation of Greeks. Greeks are the sensitivities of an option's price with respect to its underlying parameters, and accurate computation of these quantities is of central importance in options trading and risk management. We will show how a GP trained on option prices can be used to recover these sensitivities, and compare the performance of different kernel choices in doing so.

4 Computations of Greeks

4.1 Kernel comparison and Greeks

Our first implementation extends the book's example `Example-2-GP-BS-Derivatives` notebook [2]. The original implementation fits a GP with a single RBF kernel to Black-Scholes call prices as a function of the underlying asset S, and computes Delta and Vega by analytic differentiation of the RBF kernel. We extend this in three different ways.

First, six kernels are compared on the same computation: RBF (radial basis function kernel), Matérn-3/2, Matérn-5/2, Rational Quadratic, RBF+Linear (additive), and RBF+Matérn (additive). Secondly, we compute all five Black-Scholes greeks: Δ (Delta), Γ (Gamma), ν (Vega), Θ (Theta), and ρ (Rho). Lastly, for Δ we use the analytic kernel derivative formula (for RBF and Matérn-5/2) as well as central finite differences on the GP posterior mean (for composite kernels). Both methods are compared.

All experiments use the same parameters as Example 2: strike $K = 130$, rate $r = 0.002$, volatility $\sigma = 0.4$, maturity $T = 2$, with $n = 100$ training points on a rescaled domain $[0, 1]$.

4.1.1 Analytic kernel derivatives

It is known that the GP posterior mean is a weighted sum of kernel evaluations:

$$\bar{f}(x_*) = \sum_{i=1}^{n} \alpha_i \, k(x_i, x_*), \qquad \boldsymbol{\alpha} = (K + \sigma_n^2 I)^{-1} y.$$

Because $\boldsymbol{\alpha}$ does not depend on x_*, differentiation moves inside the sum:

$$\frac{\partial \bar{f}}{\partial x_*} = \sum_{i=1}^{n} \alpha_i \frac{\partial k(x_i, x_*)}{\partial x_*}, \qquad \frac{\partial^2 \bar{f}}{\partial x_*^2} = \sum_{i=1}^{n} \alpha_i \frac{\partial^2 k(x_i, x_*)}{\partial x_*^2}.$$

For the RBF kernel with length-scale ℓ:

$$\frac{\partial k_{\mathrm{SE}}}{\partial x_*}(x_i, x_*) = -\frac{x_* - x_i}{\ell^2} \, k_{\mathrm{SE}}(x_i, x_*).$$

For the Matérn-5/2 kernel with $a = \sqrt{5}|x_* - x_i|/\ell$:

$$\frac{\partial k_{5/2}}{\partial x_*} = -\sigma_f^2 \frac{5}{3\ell^2}(x_* - x_i)(1 + a)\, e^{-a},$$
$$\frac{\partial^2 k_{5/2}}{\partial x_*^2} = -\sigma_f^2 \frac{5}{3\ell^2}(1 + a - a^2)\, e^{-a}.$$

4.1.2 Results: price accuracy

Figure 7 shows the GP call price fit and absolute error for all six kernels. The Matérn kernels achieve price RMSE (root mean square error) in the range 10^{-7} to 10^{-6}, which is two to three orders of magnitude below the RBF and composite kernels, while the RationalQuadratic and composites move around 10^{-4}.

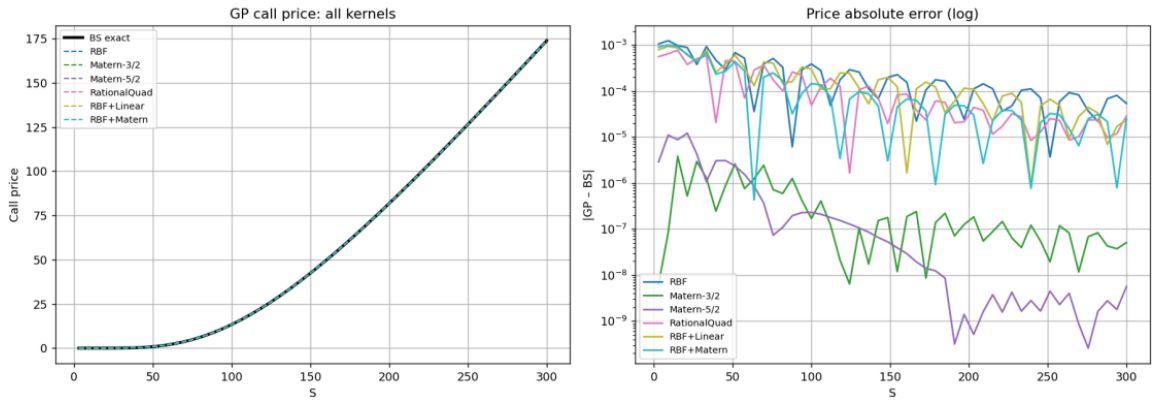

Figure 7: Black-Scholes call price: GP fit (left) and absolute error $|C_{\text{GP}} - C_{\text{BS}}|$ in log scale (right) for all six kernels. Matérn-3/2 and Matérn-5/2 are 2–3 orders of magnitude more accurate than the RBF baseline used in the book's *Example* 2.

Note that the log marginal likelihood rankings do not correspond to the RMSE rankings (see Table 1): although RationalQuadratic obtains the highest log-ML value, it only ranks third in terms of RMSE. This shows the fact that maximizing the evidence objective does not necessarily lead to the best predictive performance on unseen data.

4.1.3 Results: Delta and Gamma

Figure 8 shows Δ and Γ for all kernels, with absolute errors in log scale. Table 2 reports RMSE for all Greeks. If we analyze the graphics we can deduce some valuable insights. In first place, Matérn-3/2 achieves the best Delta (RMSE 8×10^{-7}), outperforming both RBF and the book's baseline by roughly 200×, and Matérn-5/2 achieves the best Gamma (RMSE 6×10^{-6}). It is also worth noting that the RBF+Linear kernel is not included in the Greek plots. The reason is that its linear component (DotProduct) produces very large

Kernel	Price RMSE	Price max	Log ML
RBF	3.93×10^{-4}	1.24×10^{-3}	892.9
Matérn-3/2	9.34×10^{-7}	3.80×10^{-6}	599.9
Matérn-5/2	2.83×10^{-6}	1.21×10^{-5}	837.6
RationalQuad	2.44×10^{-4}	7.73×10^{-4}	908.6
RBF+Linear	3.19×10^{-4}	9.31×10^{-4}	898.6
RBF+Matérn	2.94×10^{-4}	1.01×10^{-3}	895.7

Table 1: Price accuracy and log marginal likelihood for each kernel on the Black-Scholes call experiment ($n = 100$ training points, $K = 130$, $r = 0.002$, $\sigma = 0.4$, $T = 2$). Note: the kernel with the highest log-ML (RationalQuad) is not the most accurate pricer.

derivative values near $S = 0$ in the rescaled input domain, which makes the numerical computation of Greeks unreliable. This shows an important limitation: kernels that contain a linear term should be used carefully when the goal is to compute derivatives.

4.1.4 Results: Vega, Theta, and Rho

For Vega, Theta, and Rho, the approach is slightly different from Delta and Gamma. Instead of using the underlying price S as the input, we train a separate GP for each Greek using the relevant variable as the only input: volatility σ for Vega, time to maturity T for Theta, and interest rate r for Rho. All other parameters are kept fixed. Once each GP is trained, the corresponding Greek is obtained by taking finite differences on the GP posterior mean, exactly as for Delta and Gamma.

The results are shown in Figure 9. Matérn-5/2 produces the most accurate estimates for both Vega and Theta, achieving a Theta RMSE as low as $\sim 10^{-8}$. For Rho, the differences between kernels are very small, and the standard RBF kernel achieves the lowest error. This is because the call price varies very smoothly as a function of the interest rate, so all kernels fit it equally well and the kernel choice has almost no impact on Rho accuracy.

The logical next step, following the book, would be to compute the Greeks of the Heston model in the same analytical way, by differentiating the kernel function. However, deriving a different formula for each kernel becomes difficult and error-prone, especially for higher order Greeks, such as Gamma. For this reason, instead of differentiating the kernel analytically, we will compute the Heston Greeks using automatic differentiation. This is done in Section 4.2.4, where we reproduce the book's Heston model examples but obtaining the Greeks by autograd.

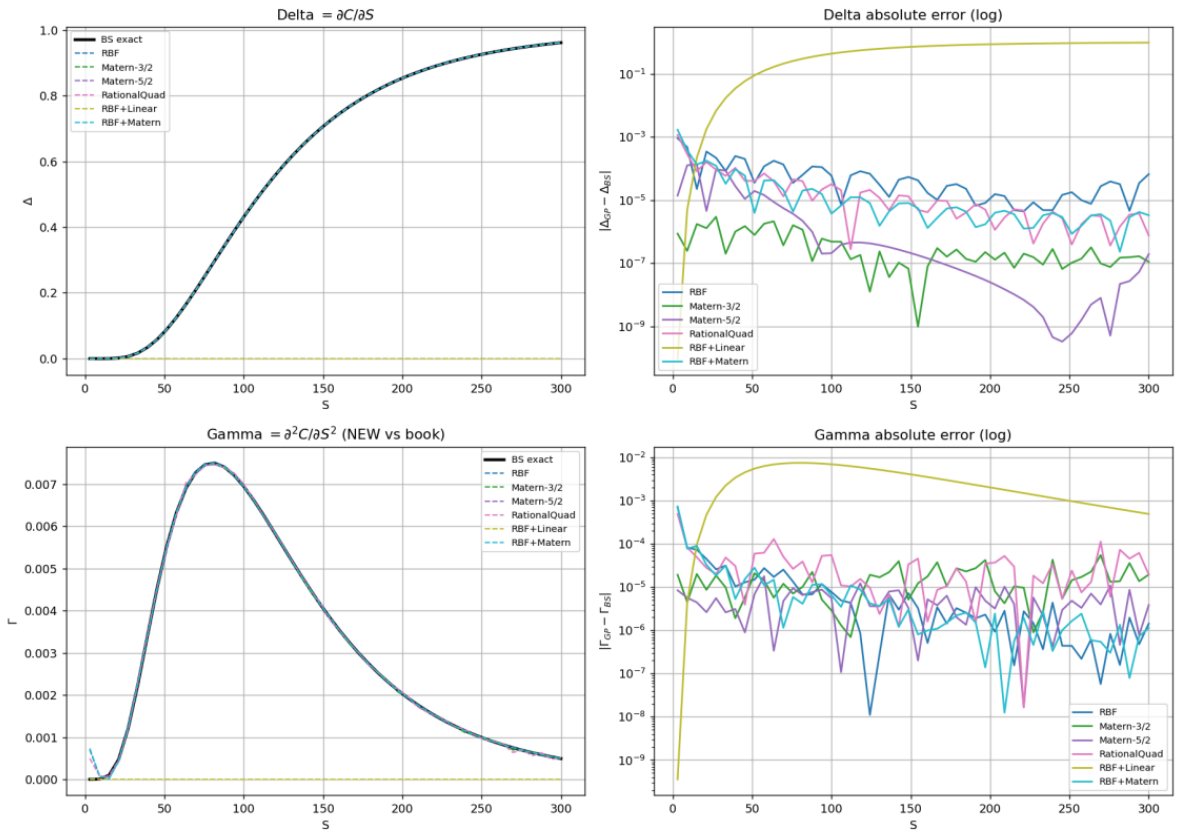

Figure 8: $\Delta = \partial C/\partial S$ (top) and $\Gamma = \partial^2 C/\partial S^2$ (bottom) for all kernels, with absolute errors in log scale. Matérn-3/2 achieves the most accurate Δ; Matérn-5/2 the most accurate Γ.

4.2 2-D GP with autograd Greeks (GPyTorch)

4.2.1 Cross-Greeks from automatic differentiation

The analytic kernel derivative approach of the previous section requires deriving and coding a separate formula for each Greek and each kernel. For first-order Greeks this is manageable, but for cross-second-order Greeks such as Vanna ($\partial^2 C/\partial S\,\partial\sigma$) the algebra is tedious and more susceptible of committing errors.

Our second implementation avoids this by training a two-dimensional GP on the surface $(S, \sigma) \mapsto C$ using `GPyTorch` [4]. Because GPyTorch is built on PyTorch, the GP posterior mean is a differentiable computational graph, and all Greeks follow from `torch.autograd.grad`, which is the same mechanism used to differentiate neural networks. Then, no kernel-derivative formula is needed at all.

4.2.2 Setup

We use a 15×15 training grid on $(S, \sigma) \in [60, 140] \times [0.1, 0.5]$ (225 points), with a Matérn-5/2 ARD kernel (separate length-scale per input dimension). Hyperparameters

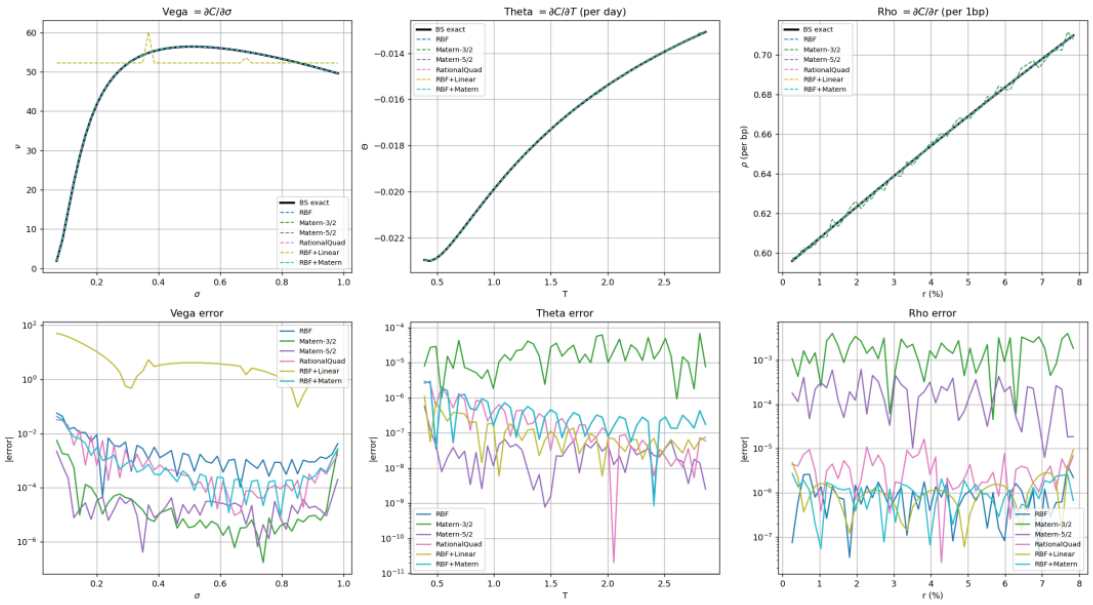

Figure 9: Vega, Theta, and Rho: GP estimates (top row) and absolute errors (bottom row) for all kernels. For Theta and Rho, all kernels achieve similar accuracy because the call price as a function of T or r is extremely smooth and all kernels fit it equally well—the error is dominated by the scale of the Greek, not the kernel choice.

Kernel	Δ	Γ	Vega	Theta	Rho
RBF	1.71×10^{-4}	1.01×10^{-4}	1.11×10^{-2}	7.7×10^{-7}	1.3×10^{-6}
Matérn-3/2	$\mathbf{8.1 \times 10^{-7}}$	2.0×10^{-5}	9.5×10^{-4}	2.7×10^{-5}	2.1×10^{-3}
Matérn-5/2	3.3×10^{-5}	$\mathbf{6.2 \times 10^{-6}}$	$\mathbf{3.3 \times 10^{-4}}$	$\mathbf{8.9 \times 10^{-8}}$	2.5×10^{-4}
RationalQuad	1.7×10^{-4}	8.1×10^{-5}	7.3×10^{-3}	7.4×10^{-7}	5.0×10^{-6}
RBF+Linear	—	—	$1.31 \times 10^{+1}$	2.3×10^{-7}	$\mathbf{2.0 \times 10^{-6}}$
RBF+Matérn	2.5×10^{-4}	1.1×10^{-4}	8.1×10^{-3}	7.7×10^{-7}	1.3×10^{-6}

Table 2: RMSE for all five Greeks across all six kernels. Bold entries are the best per column. "—" denotes kernels excluded from Greek computation due to DotProduct numerical instability.

are optimised for 200 Adam iterations by maximising the marginal log likelihood. Greeks are then extracted on a 50×50 test grid.

4.2.3 Results

Figure 10 shows the four Greek surfaces.

Table 3 summarises the errors obtained for each quantity. For first-order Greeks, the results are very accurate: the maximum relative error across the entire test grid is below 1.4% for both Δ and Vega. This means that the GP, trained on only 225 price observations, is able to recover the correct sensitivity to the underlying price and to volatility almost perfectly.

The second-order Greeks, Γ and Vanna, are less accurate, with maximum relative

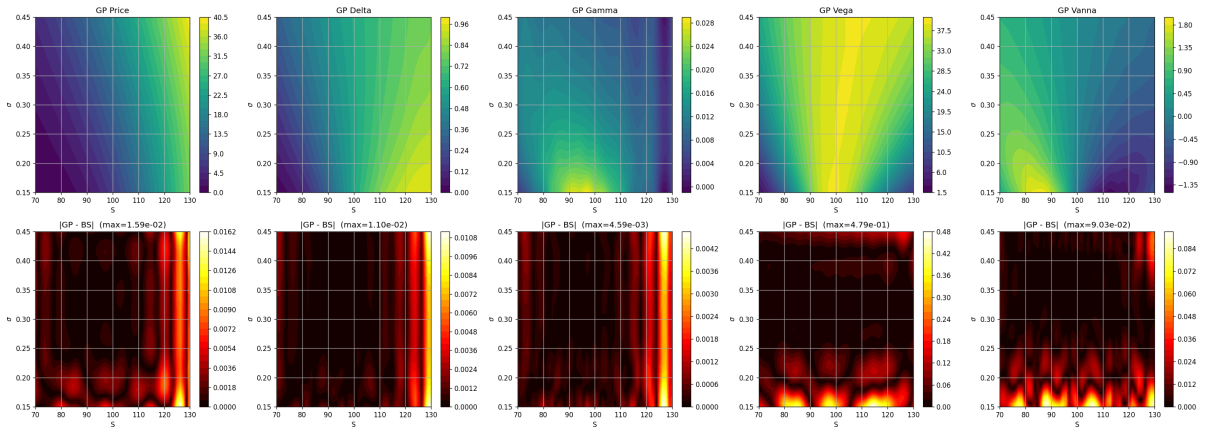

Figure 10: GP-autograd Greeks on the 2-D surface (S, σ). Top row: posterior-mean Greek surfaces. Bottom row: absolute error against BS truth. Errors concentrate near the right boundary of the training region.

errors of around 25% and 7.7% respectively. This is expected, as computing a second derivative numerically always amplifies small errors in the surface, and the GP fit is less precise near the boundaries of the training region, as can be seen in Figure 10. However, in the interior of the domain, where most real option hedging takes place, the errors are well controlled.

The most important result of this section is that Vanna is obtained completely for free, with no extra training and no need to derive any formula by hand. In the approach of the previous section, computing Vanna would require deriving the mixed partial derivative $\partial^2 k / \partial S \, \partial \sigma$ analytically for the chosen kernel, which is tedious and error-prone. With GPyTorch and automatic differentiation, it follows directly from the same idea used to compute Δ.

Quantity	Mean abs err	Max abs err	Relative max
Price	2.0×10^{-3}	1.6×10^{-2}	0.05%
Δ	1.1×10^{-3}	1.1×10^{-2}	1.32%
Γ	5.4×10^{-4}	4.6×10^{-3}	25.4%
Vega	4.2×10^{-2}	4.8×10^{-1}	1.23%
Vanna	8.9×10^{-3}	9.0×10^{-2}	7.74%

Table 3: GPyTorch autograd Greeks on the 50×50 test grid ($15 \times 15 = 225$ training points, Matérn-5/2 ARD).

4.2.4 Heston pricing and Greeks by automatic differentiation

At the beginning of Subsection 4.2 we implemented the autograd approach on the Black-Scholes model, for which we know that the Greeks have a closed form. We now apply the

same idea to the Heston model, which is the model used in the book's Example-5. As mentioned at the end of Section 4.1, the book computes the Heston Greeks by differentiating the kernel analytically. However, we instead obtain them by automatic differentiation, so we do not need to derive any analytical formula by hand.

To build the training and test data for the GP we price the European Heston call with the COS method of Fang and Oosterlee (2008) [3], which is the same pricing method used in the book. The COS method recovers the option price from the characteristic function of the log-price through a cosine series expansion, and it is both fast and very accurate.

We use the parameters of the book's Table 3.1, namely $\kappa = 0.1$, $\theta = 0.15$, $\sigma = 0.1$, $\rho = -0.9$, $r = 0.002$, $K = 100$, with initial variance $V_0 = 0.1$. We use a Matérn-5/2 kernel throughout.

First, following the book's Figure 3.6, we fit a GP to the Heston call price as a function of the underlying asset S (with $T = 2$), using $n = 50$, 100 and 150 training points drawn at random.

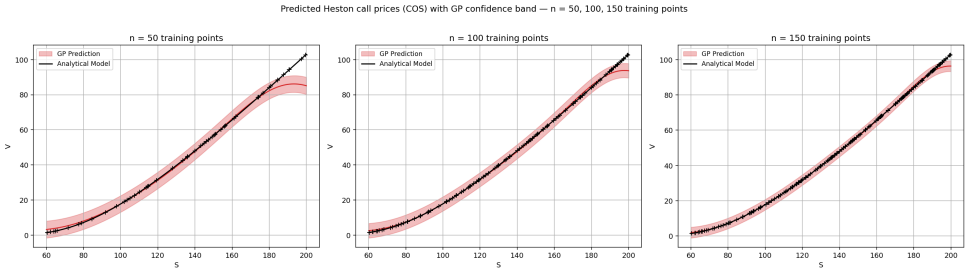

Figure 11: Predicted Heston call prices with $n = 50$, 100 and 150 training points. Black line: analytical model (COS); black "+": training points; red: GP prediction and confidence band. The band shrinks as the number of training points increases.

Figure 11 shows the result. In each panel, the black line is the analytical Heston price, the black "+" marks are the training points, and the red region is the GP prediction with its confidence band. The GP recovers the Heston price with fidelity, and as expected the confidence band becomes narrower as the number of training points grows. The largest errors appear near the right boundary of the domain, which is the same behavior observed in the book.

Next, we reproduce book's figure 3.3, which shows the call and put price surfaces over the underlying S and the volatility, at three different times to maturity. For each maturity $T - t \in \{1.0, 0.5, 0.1\}$ we fit the GP on a 30×30 grid of prices and volatilities and then we evaluate it out-of-sample on a finer 40×40 grid, so that most of the test points are

new to the model. The same Matérn-5/2 GP model is used for both the call and the put surface. The result is shown in Figure 12. The call surfaces are on the top row and the put surfaces on the bottom row, with the coloured surface being the analytical price and the black wireframe being the GP estimate. The GP estimate is practically identical to the analytical model in every panel.

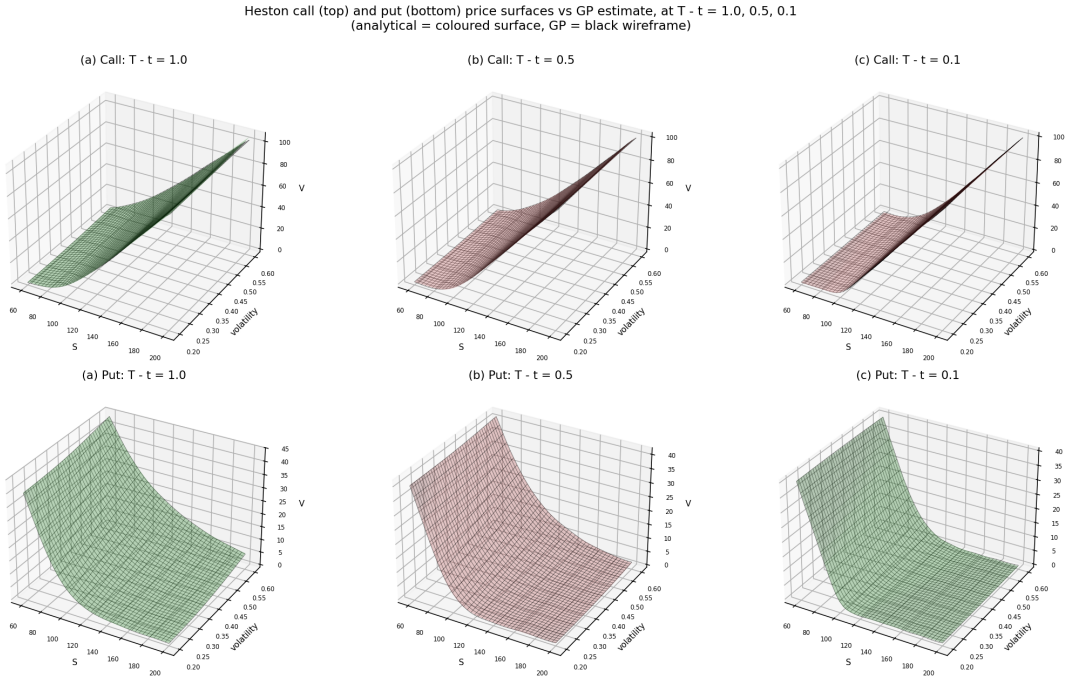

Figure 12: Heston call (top) and put (bottom) price surfaces over $(S, \text{volatility})$ at three times to maturity $T-t = 1.0,\ 0.5,\ 0.1$. The coloured surface is the analytical price (COS) and the black wireframe is the GP estimate, fitted on a 30×30 grid and evaluated out-of-sample on a 40×40 grid. The GP estimate is practically identical to the analytical model in every panel.

Finally, we compute the Greeks of the Heston call by automatic differentiation of the GP posterior mean. Since the Heston model has no closed-form Greeks, the reference values are obtained by central finite differences on the COS pricer. Figure 13 shows Delta ($\partial C/\partial S$) and Vega ($\partial C/\partial\,\text{vol}$) for $n = 100$ training points. Both Greeks are recovered with high accuracy. Notice that Vega is almost exact over the whole domain, and Delta matches almost equally in the interior, but has a small deviation near the right boundary. Then, we can conclude that the same autograd mechanism used for the Black-Scholes model works directly on the Heston model, without deriving any formula by hand.

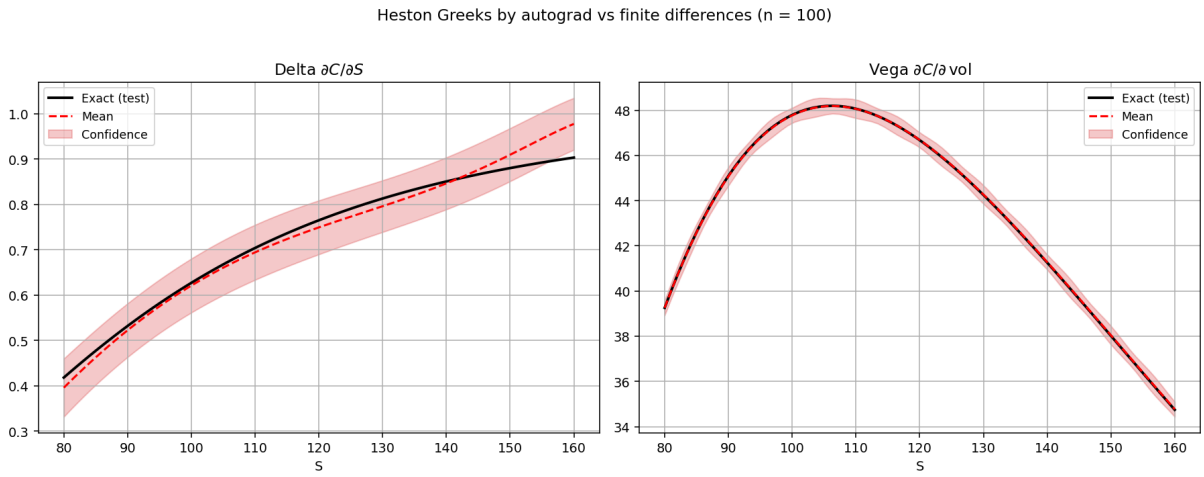

Figure 13: Heston Greeks with $n = 100$ training points. Black line: analytical reference by finite differences on the COS pricer; red dashed line: GP estimate by automatic differentiation. Left: Delta. Right: Vega.

4.3 KISS-GP scalability benchmark

4.3.1 Motivation

The book's notebooks all use $n \leq 100$ training points. In Section 2 of [2], it is explained *why* exact GP training becomes infeasible past $n \approx 10^4$ due to the $\mathcal{O}(n^3)$ Cholesky cost. Here we provide an empirical demonstration of this bottleneck and show how KISS-GP resolves it.

4.3.2 Setup

We regress the same Black-Scholes call price as a function of S on random training grids of size $n \in \{500,\ 1\,000,\ 2\,000,\ 5\,000,\ 10\,000,\ 15\,000, 20\,000, 200\,000\}$. The exact GP is implemented with `sklearn`'s `GaussianProcessRegressor`. The KISS-GP is implemented with GPyTorch's `GridInterpolationKernel` (400 inducing points on a 1-D grid) and trained for 15 Adam iterations. Wall-clock times are measured for both the fit and a 200-point prediction.

4.3.3 Results

Figure 14 shows the training time for both methods as a function of the number of training points n, using a log-log scale. Two different regimes can be clearly identified.

For small values of n (below approximately 5,000), the exact GP is actually faster than KISS-GP. This is because KISS-GP has a fixed overhead cost, as it needs to set up the sparse interpolation matrix W and run a conjugate-gradient solver regardless of how small the dataset is. For small datasets, this overhead is not justified.

However, for larger values of n, the situation changes completely. The exact GP follows the $\mathcal{O}(n^3)$ reference line closely, meaning that doubling the dataset makes it roughly eight times slower. In practice, the exact GP becomes infeasible beyond $n \sim 10^4$ due to memory limitations. KISS-GP, by contrast, grows much more slowly and continues to scale up to $n = 200{,}000$ in our experiments, running in under 3 minutes on a standard CPU. Wilson and Nickisch [12] report scaling to $n \sim 10^6$ on a single GPU.

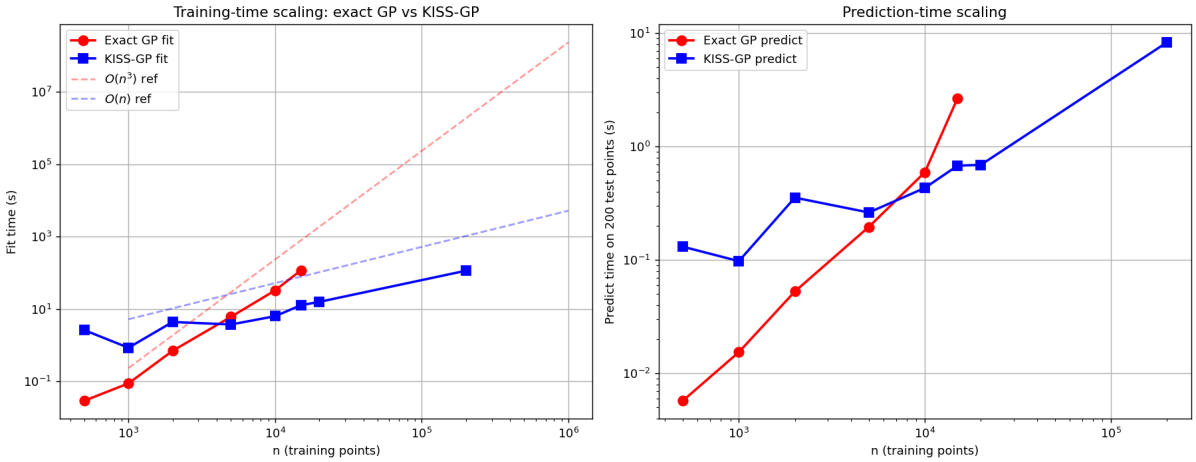

Figure 14: Training-time scaling (left) and prediction-time scaling (right) for exact GP (red) and KISS-GP (blue), on a log-log scale. The dashed lines are theoretical reference curves for $\mathcal{O}(n^3)$ and $\mathcal{O}(n)$ growth. The exact GP becomes infeasible beyond $n \approx 15{,}000$, while KISS-GP continues to scale to $n = 200{,}000$.

Figure 15 shows the quality of the KISS-GP fit at $n = 20{,}000$. The left panel compares the KISS-GP prediction (red dashed line) against the exact Black–Scholes price (black line), with a random sample of 5,000 training points shown as grey dots. The two curves are visually indistinguishable across the entire domain, which confirms that the KISS-GP prediction is a faithful approximation of the true pricing function even with such a large number of training points.

The right panel shows the absolute error on a logarithmic scale. In most of the domain the error stays around 10^{-2}, which is well within acceptable bounds for a surrogate pricer. The larger error spikes near $S \approx 0$ and $S \approx 200$ are expected, because these are the boundaries of the training domain, where the density of training points is lower and interpolation method tends to be less accurate.

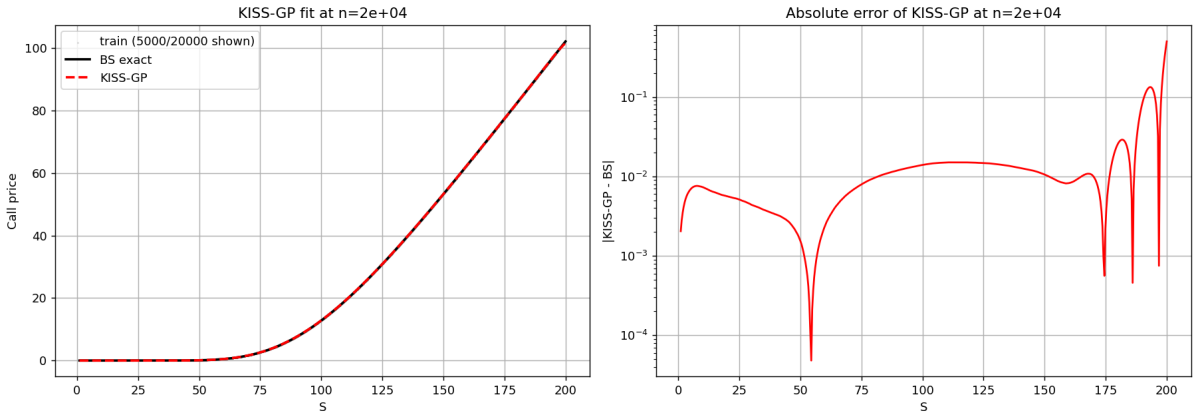

Figure 15: KISS-GP fit at $n = 2 \times 10^4$ training points. Left: KISS-GP prediction (red dashed) vs. exact Black–Scholes price (black), with 5,000 randomly selected training points shown as grey dots. Right: absolute error on a log scale.

5 Advantages, limitations and future work

Gaussian processes have several useful properties for pricing options and computing Greeks. Since the GP prediction is a smooth function of the inputs, Greeks can be computed either by differentiating the kernel by hand or, as we showed, automatically using backpropagation. This means there is no need to work out a new formula each time the kernel or model changes because the same backpropagation code that returns Delta also returns cross-Greeks such as Vanna, with no extra derivation. Another useful property is that a GP does not only give a price estimate, it also gives a confidence interval around that estimate. This is helpful in practice, since each price comes with a posterior variance, so instead of a single value we also gets a sense of how far to trust it. Our experiments show that a GP trained on as few as 225 price observations can already recover first-order Greeks with a relative error below 1.4%.

The main drawback of standard GP regression is that training is slow for large datasets. As shown in Section 4.3, the exact GP becomes too slow to use beyond roughly $n \approx 10^4$ training points. KISS-GP fixes this problem, but only works well when the number of inputs is small, since the grid it needs grows very quickly with the number of dimensions. In finance this means that using all five Black-Scholes inputs at once would already require a very large grid. We also noticed that the second-order Greeks are noisier near the edges of the training domain, where there is little data for the model to learn. Another limitation is that GPs work best when the function being learned is smooth. They do not naturally handle sharp jumps, which can appear in barrier options or digital payoffs. Finally, a high marginal log-likelihood did not always translate into better predictions in our experiments, so the two should not be treated as the same target when tuning the GP hyperparameters.

This work could be extended in several ways. The most immediate one will be to move from simulated data to real market quotes and fit the GP directly to observed option prices, to see whether the accuracy we obtained on synthetic data holds up against market noise and no-arbitrage constraints. Both standard and KISS-GP variants have already been used this way for pricing and calibration [1]. Another way, which follows naturally from the probabilistic nature of the model, is to turn the GP confidence bands into hedging-risk margins [7]. It would also be interesting to apply the GP approach to more complex products such as path-dependent or exotic derivatives, where no closed-form price exists and Monte Carlo is typically used as the benchmark. A further option is deep kernel learning, replacing the fixed kernel with a neural network feeding into a GP, which would add flexibility while keeping the uncertainty estimates. Finally, it would be useful to look at GP methods that can update their predictions as new data comes in (data assimilation), which would make the approach more suitable for real-time use.

6 Conclusion

This report looked at how Gaussian processes can be used to price financial derivatives and compute their Greeks. We extended the approach from Dixon, Halperin, and Bilokon(2020) [2] in three ways: by comparing six different kernels, by using automatic differentiation to compute Greeks instead of deriving formulas by hand, and by testing a scalable GP method called KISS-GP on much larger datasets than those used in the original work.

The first result is that the choice of kernel matters a lot. The Matérn kernels gave much better results than the standard RBF kernel used in the book. Matérn-3/2 gave the best Delta with an RMSE of 8×10^{-7}, and Matérn-5/2 gave the best Gamma and Vega. Both were roughly two orders of magnitude more accurate than RBF. We also found that the kernel with the highest marginal likelihood score, RationalQuadratic, was not the most accurate one for pricing. This shows that a higher score on the training objective does not always mean better predictions on new data. Kernels with a linear component caused problems when computing Greeks and should be avoided for that purpose.

The second result is that automatic differentiation is a practical and reliable way to compute Greeks. A GP trained on 225 price observations recovered Delta and Vega with relative errors below 1.4%. Vanna came for free, with no extra training or formulas needed. We then applied the same approach to the Heston model, which has no closed-form Greeks, and found that both Delta and Vega were recovered accurately. The fact that the same code works without any changes on a different model shows how flexible this approach is.

The third result is that KISS-GP makes it possible to train on much larger datasets. The exact GP became too slow beyond around $n = 15{,}000$ points. KISS-GP scaled up

to $n = 200{,}000$ points in under three minutes on a standard CPU, with a pricing error of around 10^{-2}, which is accurate enough for a surrogate pricer.

Overall, the results show that GPs are a good tool for derivative pricing and risk management. Using Matérn kernels, automatic differentiation for Greeks, and KISS-GP for large datasets addresses the main weaknesses of the basic GP approach and makes it practical for real financial applications.

References

[1] Jan De Spiegeleer, Dilip B Madan, Sofie Reyners, and Wim Schoutens. Machine learning for quantitative finance: fast derivative pricing, hedging and fitting. *Quantitative Finance*, 18(10):1635–1643, 2018.

[2] Matthew F Dixon, Igor Halperin, Paul Bilokon, et al. *Machine learning in finance*, volume 1170. Springer, 2020.

[3] Fang Fang and Cornelis W Oosterlee. A novel pricing method for european options based on fourier-cosine series expansions. *SIAM Journal on Scientific Computing*, 31 (2):826–848, 2009.

[4] Jacob Gardner, Geoff Pleiss, Kilian Q Weinberger, David Bindel, and Andrew G Wilson. Gpytorch: Blackbox matrix-matrix gaussian process inference with gpu acceleration. *Advances in neural information processing systems*, 31, 2018.

[5] Joan Gonzalvez, Edmond Lezmi, Thierry Roncalli, and Jiali Xu. Financial applications of Gaussian processes and Bayesian optimization. *arXiv preprint arXiv:1903.04841*, March 2019. URL `https://arxiv.org/abs/1903.04841`.

[6] Sham Kakade. Dimensionality reduction and learning: Random features and bochner's thm. Stat 991: Multivariate Analysis, Dimensionality Reduction, and Spectral Methods, Lecture 8. URL `https://homes.cs.washington.edu/~sham/courses/stat991_mult/lectures/jl_proj_margin2.pdf`. Course notes.

[7] Mike Ludkovski and Yuri Saporito. Krighedge: Gaussian process surrogates for delta hedging. *Applied Mathematical Finance*, 28(4):330–360, 2021.

[8] Ali Rahimi and Benjamin Recht. Random features for large-scale kernel machines. *Advances in neural information processing systems*, 20, 2007.

[9] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, Cambridge, MA, 2006. ISBN 026218253X. URL `http://www.GaussianProcess.org/gpml`.

[10] Astrid Schneider, Gerhard Hommel, and Maria Blettner. Linear regression analysis: Part 14 of a series on evaluation of scientific publications. *Deutsches Ärzteblatt International*, 107(44):776–782, 2010. doi: 10.3238/arztebl.2010.0776. URL `https://di.aerzteblatt.de/int/archive/article/79009`.

[11] Rens van de Schoot, Sarah Depaoli, Ruth King, Bianca Kramer, Kaspar Märtens, Mahlet G. Tadesse, Marina Vannucci, Andrew Gelman, Duco Veen, Joukje Willemsen, and Christopher Yau. Bayesian statistics and modelling. *Nature Reviews Methods Primers*, 1(1):1–26, 2021. doi: 10.1038/s43586-020-00001-2.

[12] Andrew Gordon Wilson and Hannes Nickisch. Kernel interpolation for scalable structured gaussian processes (kiss-gp), 2015. URL https://arxiv.org/abs/1503.01057.